Automatiserad rapportgenerering

Av Kevin Johansson

Syfte

Syftet med den här fördjupningen var att lära mig hur man automatiserar rapportgenerering, alltså att gå från rådata till färdig rapport i Python. Mer specifikt ville jag lära mig hur templating med **Jinja2** kan användas för att separera datalogik från presentation, och hur en HTML-mall därefter konverteras till ett PDF-format med hjälp av **WeasyPrint.** Målet var inte att göra en avancerad dataanalys, utan att bygga en fungerande, återanvändbar rapportgenerator där själva Python-fördjupningen ligger hur templating och PDF-rendering fungerar och används.

Området och dess relevans

Jag valde att fördjupa mig i automatiserad rapportgenerering – att gå från ett CSV-dataset till färdig PDF med nyckeltal, tabeller och grafer, helt automatiskt. Det kändes relevant eftersom man som data scientist sällan gör en analys bara en gång och sedan är man klar – den behöver oftast paketeras om och delas med personer som inte sitter i kod eller notebooks. Att kunna automatisera den biten sparar tid och gör att rapporterna ser likadana ut varje gång.

Viktiga begrepp

Templating betyder kort och gott att man skriver en mall (i detta fallet: en HTML-fil) med platshållare för data, som sen fylls i med riktiga värden när skriptet körs. **Jinja2** använder {{ varibel }} för att stoppa in värden och {% for %} för att loopa över listor.

Ett **filter** {{ value | currency }} omvandlar ett enskilt värde till något annat i mitt fall omvandlas en rå siffra (t.ex. 59602) till en textsträng formaterad beroende på ifall det är valuta eller ej. Vilket resulterar i att det skrivs ut som ($ 59,602).

Ett **macro** är som en funktion, fast den lever inuti HTML-mallen. Definerar den en gång och kan sedan anropa den flera gånger med olika värden. Jag använde det för mina tre KPI-boxar (försäljning, antal ordrar och snitt ordervärde), istället för att manuellt skriva HTML-koden för varje enskild KPI.

På PDF-sidan var det viktigt att förstå skillnaden mellan vanlig CSS och **CSS Paged Media (** @page **)**. Vanliga regler (body, h1, .kpi-box) styr innehållet, medan @page styr

egenskaperna på själva "pappersarket" - storlek, marginaler, bakgrund och specialgrejer som sidnummer.
En annan funktion jag använde var **running elements,** som gör att ett element kan plockas ut ur det vanliga sidflödet och istället upprepas automatiskt på flera sidor. I mitt fall använde jag det för att få en avgränsande linje och datumraden att synas överst på alla sidor **efter den första** utan att rubriken hänger med.

Genomförande

Bibliotek: pandas för att läsa in och räkna på datan, **matplotlib** för graferna, **Jinja2** för HTML-templatingen och **WeasyPrint** för att göra om HTML till PDF.

Data: Jag valde ursprungligen hela Superstore Sales Dataset (2015-2018), men valde senare att bara jobba med 2018 års data och delade upp den i fyra kvartalsvisa CSV-filer. Anledningen var dels att göra scenariot mer verklighetsförankrat men också att ha flera filer att testa och jämföra resultaten. Vilket kvartal som ska användas väljs genom att peka DATA_PATH i main.py mot rätt fil (t.ex. sales_2018_Q3.csv*),* så samma skript kan köras om på valfritt kvartal utan att resten av koden behöver ändras.

Struktur och tekniska val: Jag delade upp lösningen i separata moduler under en utils/-mapp istället för att ha all kod i en enda fil:

- data_loader.py - läser in och tolkar CSV-filen
- metrics.py - beräknar nyckeltal (total försäljning, antal ordrar, snitt ordervärde, topp 5-produkter, försäljning per kategori/region samt veckovis trend)
- visualizations.py - ansvarar för att generera graferna
- report_generator.py - renderar Jinja2-mallen och skriver ut PDF:en

Samma tankesätt applicerade jag även på mall-sidan: jag la mitt KPI-macro i en egen fil (macros.html) istället för att ha det inline i report_template.html, och importerar det istället där det behövs. Jag har bara ett macro just nu, men principen är densamma som modulariseringen i Python-koden – och blir mer värdefull om fler macron läggs till efter hand som projektet växer.

main.py binder ihop allt i ett litet, läsbart flöde. Den här uppdelningen gör det tydligt var man ska ändra något - vill man lägga till ett nytt nyckeltal görs det i metrics.py, en ny graf i visualisation.py, utan att röra resten av koden.

Jag körde på mörkt tema för rapporten med en färgpalett jag återanvänder i både grafer och HTML.

Jag valde att byta från månads- till veckovis trend när jag minskade dataunderlaget till kvartalsvis för att få en tydligare lägesbild.

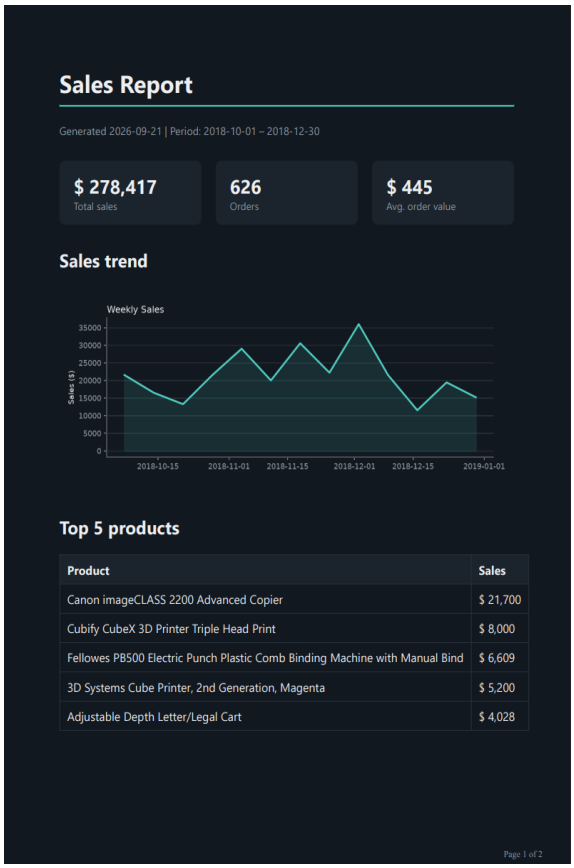
Saker som strulade, och hur jag löste dem:

- **WeasyPrint** behöver **Pango** på Windows (ett bibliotek för textlayout/typsättning) för att kunna rendera text i PDF:en, vilket inte följde med pip install weasyprint automatiskt. Men via deras dokumentation hittade jag lösningen genom att ladda hem **MSYS2** och installera **Pango** via den terminalen.
- **Windows 11 Smart App Control** hindrade mig från att köra skripet eftersom det klagade på en osignerad DLL, vilket jag löste genom att stänga av funktionen.
- **Vit kant runt PDF:en.** Bakgrunden på body täcker bara innehållet, inte marginalen mot PDF:ens kant, vilket jag efter många om och men löste genom att sätta bakgrundsfärgen direkt på @page istället.

Resultat

Slutresultatet är ett skript man kör med python main.py, som tar en rå CSV-fil och genererar en färdig report.pdf med:

- En KPI-del med total försäljning, antal ordrar och snitt ordervärde
- En graf över veckovis försäljningstrend
- En tabell med de fem mest sålda produkterna
- Grafer över försäljning per kategori och region





Begränsningar och möjliga förbättringar

Lösningen är medvetet avgränsat och har flera begränsningar:

- Den hanterar bara en CSV-fil med ett fast, förväntat kolumnschema – byter man dataformat behöver koden justeras manuellt.
- Det finns ingen felhantering för dålig eller ofullständig indata – skriptet förutsätter att kolumnerna finns och är korrekt formaterade.
- Kopplingen mellan metrics.py och mallen är inte automatisk - lägger man till ett nytt nyckeltal i Python måste man även lägga till det i HTML-mallen.

Således finns det massor av möjliga förbättringar som:

- Datavalidering och felhantering
- Stöd för fler datakällor
- Fler exportformat som Excel eller JSON
- Pytest för att säkerställa att beräkningarna faktiskt ger rätt resultat

Koppling till yrkesrollen

Automatiserad rapportgenerering är rätt direkt användbart för en data scientist - återkommande rapportering är vanligt, veckovisa eller månatliga sammanfattningar till folk som själva inte kodar. Istället för att bygga om samma analys i Excel eller en notebook varje gång ny data kommer in kan man köra samma skript om och om igen och alltid få samma, konsekventa resultat.

Källor

- Jinja2, dokumentation: https://jinja.palletsprojects.com/en/stable/
- WeasyPrint, dokumentation: https://doc.courtbouillon.org/weasyprint/stable/
- Superstore Sales Dataset, Kaggle: https://www.kaggle.com/datasets/rohitsahoo/sales-forecasting

Självreflektion

1. **Vad lärde du dig som du inte kunde innan?**
   Grunderna i Jinja2 – hur man håller isär data och presentation med filter och macros – och hur man faktiskt går från HTML, till en riktigt PDF med WeasyPrint.

2. **Vad var svårast att förstå?**
   Det som orsakade mest huvudvärk var att förstå skillanden mellan vanlig CSS och @page CSS vilket orsakade flera fel innan jag förstod varför.

3. **Vilket tekniskt val är du mest nöjd med och varför?**
   Att dela upp Python-koden i moduler istället för att ha allt i en och samma fil som jag började med. Bröt ut återanvändbara Jinja2-macron till en egen fil istället för att ha allt i mallen, för bättre struktur ifall fler macron tillkommer.

4. **Vad hade du gjort annorlunda om du började om?**
   Det hade nog varit att börja jobba strukturerat med moduler från start istället för att justera det i efterhand när det började stökigt.

5. **Vad skulle varit ett naturligt nästa steg om du fortsatte?**
   Lägga till validering av indata, göra tidsperiod/datakälla konfigurerbart, ett enkelt användargränssnitt samt skriva enklare pytest så rapporten inte går sönder om datan ändras.

6. **Vilket betyg tycker du själv att arbetet motsvarar – G eller VG?**
   Jag tycker att arbetet uppfyller G-kraven tydligt: jag har valt och avgränsat ett relevant område, satt mig in i den officiella dokumentationen (Jinja2 och WeasyPrint), byggt en fungerande lösning, och kan förklara både koden och de tekniska begreppen bakom den – filter, macro, @page, running elements. Koden är uppdelad i moduler och dokumenterad med README.
   Jag har gjort val som jag kan motivera, t.ex. att bryta ut macro till en egen fil, byta till veckovis trend när datamängden krympte.